Prosit 7: SYSTÈME DE FICHIER

Analyse de la situation

1. Le Contexte

Le système embarqué THS réalise l'acquisition continue de données provenant de multiples capteurs. Chaque relevé de mesure génère une ligne de texte formatée (ex: CSV) comprenant :

- L'horodatage (Date & Heure via le composant RTC de la STM32)
- L'identifiant du capteur
- La valeur mesurée
- L'état du système et les séparateurs

Taille maximale d'une ligne de mesure : 256 octets.

2. Le Dilemme du Stockage (Temps vs Espace)

Sur un système embarqué, la gestion de la mémoire fait face à deux contraintes antagonistes :

1. **La fragmentation interne (Gaspillage d'espace) :** Si les fichiers créés sont très petits par rapport à la taille d'une unité d'allocation (cluster), une grande partie du secteur mémoire est allouée dans le vide.
2. **La complexité d'accès (Perte de temps) :**
    - Si toutes les données sont écrites dans un unique fichier géant, la recherche d'une mesure spécifique nécessite un parcours séquentiel long et gourmand en ressources.
    - Si l'on crée un fichier par mesure, la carte SD se remplit de structures de répertoires lourdes et inefficaces.
    - Si l'on utilise un fichier d'index dédié, toute coupure d'alimentation inopinée risque de corrompre l'index et de rendre les données inaccessibles.

Etude des ressources

1. Pourquoi un système de fichiers ?

Un disque dur ou un SSD conserve les données même lorsque l'ordinateur est éteint : c'est une mémoire de masse non volatile. Mais les données y sont enregistrées sous forme binaire. Le

système d'exploitation a donc besoin d'une méthode pour les organiser, les localiser et gérer leur utilisation : c'est le rôle du système de fichiers.

À retenir : le support stocke les bits ; le système de fichiers donne une organisation exploitable à ces bits.

2. Fichiers, attributs et formats

Un fichier est une unité logique de données identifiée par un nom. Le système de fichiers conserve des métadonnées (ou attributs), par exemple la taille, les dates et les droits d'accès. Selon le système, d'autres informations peuvent être enregistrées.

Le format décrit la manière dont le contenu est structuré et interprété : TXT pour du texte simple, JPEG pour une image, WAV pour de l'audio, PDF pour un document, etc. L'extension aide à reconnaître le format, mais elle ne garantit pas à elle seule le contenu réel du fichier.

Distinction importante : le système de fichiers gère l'emplacement, le nom et les métadonnées ; l'application comprend généralement la structure interne du contenu (par exemple, un lecteur PDF interprète un PDF).

3. Répertoires : une organisation en arbre

Un répertoire (ou dossier) regroupe des fichiers et peut contenir d'autres répertoires. Cette organisation forme une hiérarchie en arbre, qui part d'une racine. Un chemin permet de retrouver un élément dans cette hiérarchie, par exemple : /Documents/Cours/notes.pdf sous un système de type Unix.

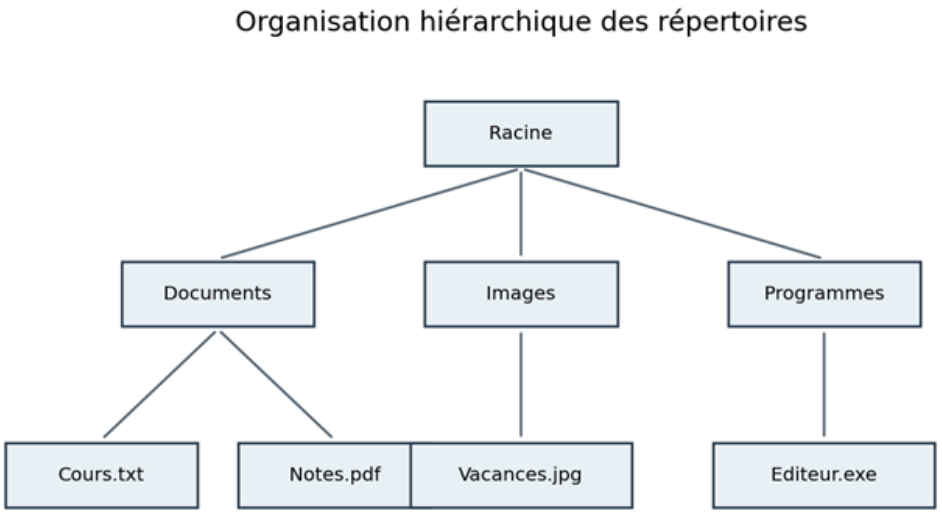


Schéma pédagogique : dossiers et fichiers organisés en arbre.

4. Comment le stockage est-il organisé ?

Les supports sont découpés en unités adressables, souvent appelées secteurs. Les systèmes de fichiers regroupent fréquemment plusieurs secteurs en clusters (unités d'allocation). Un fichier peut occuper un ou plusieurs clusters ; le système doit donc mémoriser quels blocs lui appartiennent.

LBA (Logical Block Addressing) désigne un mode d'adressage logique des blocs du support. Le système de fichiers utilise les informations d'allocation pour retrouver les blocs qui composent chaque fichier.

5. Principales méthodes d'allocation

A. Allocation contiguë

Les blocs d'un fichier sont placés les uns à la suite des autres. Le système peut retenir le premier bloc et la longueur du fichier. L'accès séquentiel est efficace, mais la suppression et l'ajout de fichiers peuvent laisser de petits espaces libres dispersés, insuffisants pour certains fichiers : c'est la fragmentation externe.

B. Allocation par chaîne de blocs

Les blocs peuvent être dispersés ; chaque bloc indique où se trouve le suivant. Cela facilite l'utilisation des espaces libres, mais accéder directement à un bloc situé loin dans le fichier peut demander de suivre la chaîne depuis le début.

C. FAT (File Allocation Table)

La FAT conserve dans une table les liens entre les clusters, plutôt que de devoir lire chaque lien directement sur le disque. Elle facilite le suivi de la chaîne, mais la table occupe de la mémoire et doit être gérée de façon fiable.

D. i-nodes

Dans les systèmes de type Unix, un i-node contient des métadonnées et des références vers les blocs de données du fichier. Des mécanismes d'indirection permettent de gérer les fichiers plus volumineux. Le nom du fichier est généralement enregistré dans le répertoire, qui associe ce nom à l'i-node.

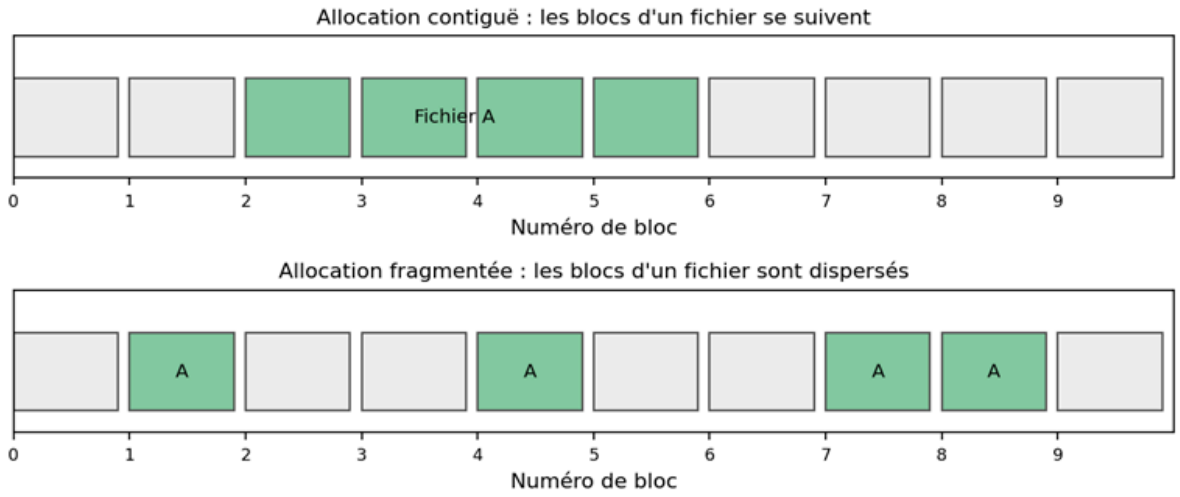


Illustration simplifiée : blocs contigus et blocs dispersés (fragmentation).

6. Fragmentation et défragmentation

La fragmentation apparaît lorsqu'un fichier est réparti en plusieurs zones non contiguës. Sur un disque dur mécanique, regrouper les blocs peut réduire les déplacements de la tête de lecture et améliorer certaines performances. La défragmentation réorganise les blocs pour les rapprocher. Sur un SSD, le bénéfice est généralement faible et les écritures supplémentaires sont à éviter : il faut privilégier les outils d'optimisation prévus par le système.

Attention : la fragmentation n'est pas la même chose que le manque d'espace disque. Elle concerne la disposition des blocs, pas simplement la quantité d'espace disponible.

7. Ouverture et accès à un fichier

Lorsqu'un programme veut utiliser un fichier, il demande au système d'exploitation de l'ouvrir. Le système vérifie notamment que le fichier existe et que le processus possède les droits nécessaires. Si l'accès est autorisé, le programme peut lire ou écrire ; lorsqu'il a terminé, il ferme le fichier. Le système d'exploitation fait le lien entre l'opération demandée et les blocs réellement stockés sur le support.

Cette interface permet aux applications de manipuler des fichiers sans avoir à gérer elles-mêmes les détails physiques du disque.

8. Tableau récapitulatif

Méthode	Principe	Point à retenir
Contiguë	Blocs consécutifs	Rapide, mais risque de fragmentation externe
Chaînée	Chaque bloc pointe vers le suivant	Souple, accès direct moins pratique
FAT	Liens regroupés dans une table	Suivi des clusters plus simple, table à maintenir

i-node	Métadonnées + références aux blocs	Courant dans les systèmes de type Unix

9. À retenir

- Un système de fichiers organise les données et permet de les retrouver.
- Un fichier possède un nom et des métadonnées ; son format décrit la structure de son contenu.
- Les répertoires forment une hiérarchie, généralement représentée comme un arbre.
- Les données sont allouées en blocs/secteurs ou en clusters.
- Contiguë = blocs voisins ; chaîne = liens entre blocs ; FAT = table de liens ; i-node = métadonnées et références aux blocs.
- Le système d'exploitation vérifie les droits avant de donner accès à un fichier.

II. COMPRENDRE LE SYSTÈME DE FICHIERS FAT32 ET LE FORMATAGE

1. Qu'est-ce que le système FAT32 ?

Le système FAT32 (File Allocation Table 32-bits) organise le stockage physique sous forme de blocs d'octets regroupés en clusters (ou unités d'allocation).

- La taille d'un cluster est fixée lors du formatage.
- **Règle fondamentale :** Un cluster est la plus petite unité d'espace pouvant être attribuée à un fichier. Même si un fichier ne contient qu'un seul octet, il occupe la totalité d'un cluster.

2. Analyse de l'erreur du réglage à 64 ko (Proposition initiale)

Si l'on configure la carte avec des clusters de 64 ko (65 536 octets) :

- Un fichier de 1 mesure (256 octets) occupera physiquement 64 ko sur la carte.
- **Taux de perte (gaspillage) :** environ 99,61% d'espace gâché.

3. Solution de Formatage Recommandée

Il faut formater la carte MicroSD avec des clusters de 4 ko (4 096 octets) ou 8 ko (8 192 octets).

Commande de formatage sous Linux :

`sudo mkfs.vfat -F 32 -s 8 /dev/sdX1`

(Le paramètre `-s 8` définit 8 secteurs de 512 octets par cluster, soit 4 096 octets).

ÉVALUATION COMPARATIVE DES ARCHITECTURES DE FICHIERS

Architecture	Complexité Écriture	Complexité Recherche	Sécurité Coupure Énergie	Gaspillage Mémoire Disque
Option A : Liste chaînée `LOG0001.CSV`	**O(1)** Écriture directe en fin de fichier	**O(N)** Lecture séquentielle requise	**Élevée** Seul le dernier fichier est impacté	**Variable** Selon la fréquence de coupure
Option B : Fichier d'indexation central `INDEX.DAT`	**O(1)** Nécessite 2 écritures (données + index)	**O(1) / O(log N)** Accès direct par lecture d'index	**Très faible** Risque élevé de corruption d'index	**Faible** Structuration optimisée
Option C : Arborescence horodatée *(Solution retenue)*	**O(1)** Écriture directe dans la tranche	**O(1)** Accès calculé par le nom du chemin	**Élevée** Structure isolée par tranche horaire	**Minimal** Gaspillage maîtrisé et sous contrôle

ARCHITECTURE ET LIVRABLE TECHNIQUE RETENU

1. Structure Recommandée (Indexation Implicite)

Au lieu de maintenir un fichier d'index en mémoire, l'indexation est implicite et découle directement de l'horloge temps réel (RTC) de la STM32 :

```
SD_CARD/
└── 2026_04_27/          ← Répertoire quotidien (Format : YYYY_MM_DD)
    ├── H00_00.CSV       ← Fichier de la tranche 00h00 - 00h59
    ├── H01_00.CSV       ← Fichier de la tranche 01h00 - 01h59
    ├── ...
    └── H23_00.CSV       ← Fichier de la tranche 23h00 - 23h59
```

2. Formats des Données (.CSV)

Chaque ligne du fichier respecte la structure fixe suivante :

```
TIMESTAMP;ID_CAPTEUR;VALEUR;UNITE;STATUT_SYSTEME
2026-04-27T14:15:30Z;TEMP_01;23.45;CELSIUS;OK
```

CALCULS DE DIMENSIONNEMENT ET CAPACITÉ

1. **Fréquence d'échantillonnage standard :** 1 mesure / 10 secondes (6 mesures / minute).
2. **Volume horaire :** 360 mesures/heure × 256 octets = **92 160 octets** (environ 90 ko / fichier).
3. **Volume quotidien :** 8 640 mesures/jour × 256 octets = **2,21 Mo / jour**.
4. **Calcul de l'espace occupé avec des clusters de 4 ko :**
    - Un fichier de 90 ko occupe **23 clusters de 4 ko** (94 208 octets attribués).
    - Perte par fichier : **environ 2 ko seulement**.
5. **Autonomie d'une carte MicroSD 16 Go FAT32 :**
    - $\text{Autonomie} = \frac{16,000\text{ Mo}}{2{,}21\text{ Mo/jour}} \approx 7,239\text{ jours}$ (environ **19 ans** de stockage continu).

GUIDE D'IMPLÉMENTATION CODE SUR STM32 (FatFs)

1. Optimisation du Tampon d'Écriture en RAM

Pour éviter d'user prématurément la carte MicroSD et accélérer les cycles d'écriture, les mesures ne doivent pas être inscrites une par une sur la carte. Elles sont accumulées dans un tampon (buffer) en mémoire RAM SRAM2 de la STM32L476RG.

- **Taille optimale du buffer RAM :** 2 048 octets (soit 8 lignes de 256 octets ou 4 secteurs SD).
- **Déclenchement d'écriture physique :** Dès que le buffer est plein ou toutes les X minutes via un appel à `f_sync()`.

2. Algorithme d'accès direct O(1)

Lorsqu'un utilisateur recherche une donnée pour le **27 Avril 2026 à 14h15** :

1. Le système ne lit pas la carte pour chercher où se trouve la donnée.
2. Le programme génère directement le chemin d'accès textuel : `"/2026_04_27/H14_00.CSV"`.
3. Il ouvre le fichier immédiatement via `f_open()`.
4. La recherche est effectuée en temps constant **O(1)**.

L'Accès Direct $O(1)$ (Nommage dynamique du chemin)

Au lieu de parcourir toute la carte SD pour trouver où écrire, cette fonction construit directement le nom du dossier et du fichier à partir de la date et de l'heure.

```c
static void Generate_FilePath(char *pathBuffer, uint16_t year, uint8_t month, uint8_t day, uint8_t hour) {
    // Génère un chemin du type : "/2026_04_27/H14_00.CSV"
    sprintf(pathBuffer, "/%04d_%02d_%02d/H%02d_00.CSV", year, month, day, hour);
}
```

Le Tampon en RAM (Évite de gâcher l'espace et préserve la carte SD)

Au lieu d'écrire chaque mesure de 256 octets directement sur la SD (ce qui ralentit le système et use la carte), on accumule les lignes dans une mémoire tampon de **2 048 octets**.

```c
// Si le tampon en SRAM risque de déborder, on lance l'écriture sur le disque
if (bufferOffset + lineLen >= SD_BUFFER_SIZE) {
    fr = SD_Logger_FlushBuffer();
    if (fr != FR_OK) return fr;
}

// Sinon, on copie simplement la ligne en RAM
memcpy(&sdBuffer[bufferOffset], tempLine, lineLen);
bufferOffset += lineLen;
```

La Sécurité contre les Coupures de Courant (`f_sync`)

Pour éviter de perdre des données si l'alimentation est coupée brutalement, la fonction d'écriture appelle `f_sync()`. Cela valide physiquement l'écriture sur la carte microSD sans avoir à fermer et rouvrir le fichier à chaque fois.

```c
FRESULT SD_Logger_FlushBuffer(void) {
    if (bufferOffset == 0) return FR_OK;

    // Écrit le contenu du tampon RAM vers le fichier FatFs
    FRESULT fr = f_write(&logFile, sdBuffer, bufferOffset, &bytesWritten);
    if (fr == FR_OK) {
        f_sync(&logFile); // Force la sauvegarde physique sur la carte SD
        bufferOffset = 0; // Réinitialise le tampon
    }
    return fr;
}
```

Le Changement Automatique de Fichier (Gestion horaire)

Dès que la tranche horaire change (par exemple à 15h00), le code détecte que le chemin cible n'est plus le même. Il vide le buffer, ferme le fichier courant et ouvre le nouveau en mode ajout (`FA_OPEN_APPEND`).

```c
if (strcmp(targetPath, currentFilePath) != 0) {
    SD_Logger_FlushBuffer();  // Vide les dernières données restantes
    f_close(&logFile);        // Ferme le fichier de l'heure précédente

    Ensure_DirectoryExists(meas->year, meas->month, meas->day); // Crée le dossier du jour si besoin
    f_open(&logFile, targetPath, FA_OPEN_APPEND | FA_WRITE);    // Ouvre le nouveau fichier horaire
}
```